

# Teach Module 47: Professional Skills: Communication, Collaboration, and Ownership

This note is based on Module 47's topic title from the bootcamp outline, but the teaching content is original and does not use the course material as the lesson source.

## Module 47 Overview

Module 47 is about becoming the kind of engineer teams trust: someone who can explain decisions, collaborate without ego, document clearly, accept feedback, and own work from idea to production.

The main topics are:

- Technical documentation and architecture diagrams
- Explaining technical tradeoffs to stakeholders
- Pair programming and cross-team collaboration
- Giving and receiving constructive feedback
- Owning features end-to-end

## Technical Documentation

Good documentation is not "write everything down." Good documentation helps someone make progress without needing a meeting.

As a software engineer, you usually write several types of documentation.

### README

A README answers:

```
What is this project?
How do I run it?
How do I test it?
What environment variables do I need?
Where are the important files?
```

A strong README is practical:

```
# Order Service

This service handles order creation and order status updates.

## Run locally

1. Start PostgreSQL
2. Set DATABASE_URL
3. Run:

mvn spring-boot:run

## Test
```

```
mvn test
```

## Architecture Notes

Architecture documentation explains how the system is shaped.

It should answer:

```
What are the major components?
How do they communicate?
Where is data stored?
What external systems are involved?
What tradeoffs were chosen?
```

Example:

```
The frontend calls the Spring Boot API.
The API validates the request and stores order data in Oracle.
When an order is created, the API publishes an event to Kafka.
Other services consume that event asynchronously.
```

## Decision Records

A decision record explains why a technical choice was made.

Simple format:

```
# Decision: Use Kafka for order events

## Context
Multiple services need to react when an order is created.

## Decision
Use Kafka to publish order-created events.

## Why
Kafka supports asynchronous communication and allows services to evolve independently.

## Tradeoffs
This adds operational complexity and requires monitoring consumer lag.
```

The goal is not fancy writing. The goal is future clarity.

## Architecture Diagrams

A diagram should make relationships obvious.

A good software diagram usually shows:

Components  
Data flow  
External systems  
Boundaries  
Direction of communication

Example:

```
User
|
v
React Frontend
|
v
Spring Boot API
|
+--> Oracle Database
|
+--> Kafka Topic
      |
      v
      Notification Service
```

When drawing diagrams, avoid stuffing in every class and method. Architecture diagrams are not code dumps. They are maps.

Useful diagram types:

| Diagram Type           | Use It For                                |
|------------------------|-------------------------------------------|
| System context diagram | Showing users and external systems        |
| Container diagram      | Showing apps, services, databases, queues |
| Sequence diagram       | Showing order of interactions             |
| Deployment diagram     | Showing where things run                  |
| ER diagram             | Showing data relationships                |

A good diagram should answer one question well.

Bad diagram question:

Can this show everything?

Good diagram question:

Can this help someone understand how order creation works?

## Explaining Technical Tradeoffs

A tradeoff means you are choosing one benefit while accepting one cost.

Technical decisions are rarely "good vs bad." They are usually:

```
Simple now vs flexible later
Fast delivery vs long-term maintainability
Performance vs readability
Security vs convenience
Consistency vs availability
```

Example:

Option A: Synchronous REST call

Pros:

- Simple to understand
- Easier to debug
- Immediate response

Cons:

- Services are tightly coupled
- If the downstream service is down, the request may fail

Option B: Kafka event

Pros:

- Services are less coupled
- Better for asynchronous workflows
- More resilient when consumers are temporarily unavailable

Cons:

- More infrastructure
- Harder debugging
- Requires monitoring and retry handling

When speaking to stakeholders, do not drown them in implementation detail.

Instead of:

```
We need Kafka because the consumer offsets and broker partitions allow decoupled
asynchronous event propagation.
```

Say:

```
Kafka lets us process work in the background, so one system can continue accepting orders
even if another system is temporarily slow. The tradeoff is that it adds monitoring and
```

```
operational complexity.
```

Strong engineers translate complexity without distorting it.

## Pair Programming And Collaboration

Pair programming means two people work on the same problem together.

Common roles:

```
Driver: writes the code
Navigator: reviews direction, spots issues, thinks ahead
```

The roles should switch regularly.

Good pair programming sounds like:

```
"What do you think this method should return?"
"Let's write the failing test first."
"I think this condition misses the null case."
"Can you explain why you chose that approach?"
```

Bad pair programming sounds like:

```
"Move over, I'll do it."
"That's wrong."
"Just type what I say."
```

Pairing is not about proving who is smarter. It is about improving the solution while sharing context.

Cross-team collaboration is similar. Backend, frontend, QA, DevOps, security, and product teams often see different risks.

A backend engineer may care about API design.

A frontend engineer may care about loading states and error messages.

A QA engineer may care about edge cases.

A DevOps engineer may care about deployment, observability, and rollback.

A security engineer may care about authentication, authorization, secrets, and input validation.

Professional collaboration means you treat these perspectives as part of the engineering work, not interruptions from the real work.

## Giving Constructive Feedback

Good feedback is specific, respectful, and useful.

Weak feedback:

This code is bad.

Better feedback:

This method is doing validation, database access, and response formatting. Could we split it so the controller delegates business logic to a service?

Weak feedback:

This design is confusing.

Better feedback:

I had trouble understanding how errors move from the API to the frontend. Could we add a short sequence diagram or document the error response format?

A helpful feedback pattern:

Observation: What you noticed  
Impact: Why it matters  
Suggestion: What could improve it

Example:

I noticed this endpoint returns different error shapes depending on the failure. That may make frontend handling harder. Could we standardize errors with a common response object?

## Receiving Feedback

Receiving feedback well is a serious professional skill.

Your first instinct may be to defend your work. That is normal. But good engineers learn to slow down.

Useful responses:

"Good catch."  
"I see what you mean."  
"Can you show me the case you're worried about?"  
"Let me think through that tradeoff."  
"I agree. I'll update it."

If you disagree, do it with reasoning:

I see the concern. I chose this approach because the endpoint is only used internally and the simpler design avoids extra abstraction. But if we expect more consumers soon, I agree the abstraction would help.

That kind of response shows maturity. You are not blindly accepting feedback, but you are not protecting your ego either.

## Ownership

Owning a feature end-to-end means you do not stop at "the code compiles."

Ownership includes:

```
Understanding the requirement
Clarifying edge cases
Designing the solution
Writing the code
Testing it
Reviewing logs and errors
Documenting usage
Supporting deployment
Responding to bugs
Improving after feedback
```

Example: suppose you build "Create Order."

A non-owner mindset says:

```
I finished the controller.
```

An owner mindset says:

```
The create-order flow works from frontend to backend, validates bad input, stores the
record, publishes the event, has tests, logs failures clearly, and has README instructions
for local testing.
```

That is the difference.

## Practice Exercises

### Exercise 1: Write A Technical README

Pick a small Java/Spring Boot project and write a README with:

```
Project purpose
Tech stack
How to run locally
Required environment variables
How to run tests
Common troubleshooting notes
```

Practice goal: make the project understandable to another developer in under 5 minutes.

### Exercise 2: Create An Architecture Diagram

Draw a simple system diagram for an app like:

```
React frontend
Spring Boot backend
Oracle database
Kafka topic
External payment API
```

Show arrows for request flow and event flow.

Practice goal: explain how the system works without showing code.

### Exercise 3: Write A Technical Decision Record

Write a short decision record for one choice:

```
Use Kafka vs REST
Use Oracle vs PostgreSQL
Use JWT vs session-based auth
Use monolith vs microservices
```

Use this structure:

```
Context
Decision
Reasons
Tradeoffs
Risks
Alternatives considered
```

Practice goal: explain why a decision was made, not just what was chosen.

### Exercise 4: Explain A Tradeoff To Two Audiences

Choose one technical topic, for example:

```
Caching
Kafka
Microservices
Automated testing
CI/CD
```

Explain it twice:

1. To a developer
2. To a non-technical stakeholder

Practice goal: adjust your language based on the audience.

### Exercise 5: Code Review Feedback Practice

Look at a sample controller that contains too much logic, then write 3 review comments.



Example feedback topics:

```
Validation belongs in a separate layer
Business logic should move to a service
Error responses should be consistent
Tests are missing for failure cases
Method names could be clearer
```

Practice goal: make feedback specific, respectful, and actionable.

## Exercise 6: Receive Feedback Professionally

Pretend someone reviewed your code and said:

```
This method is too long.
The naming is confusing.
There are no tests for invalid input.
This design may not scale.
```

Write professional responses for each one.

Example:

```
Good point. I'll split the validation and persistence logic into separate methods and add
tests for invalid input.
```

Practice goal: respond without defensiveness.

## Exercise 7: Pair Programming Simulation

With a partner, solve a small coding task:

```
Build a Java method that validates an order request.
```

One person is the driver, the other is the navigator. Switch roles after 10 minutes.

Practice goal: communicate while coding, not after coding.

## Exercise 8: Feature Ownership Checklist

Choose a feature, such as:

```
User registration
Create order
Submit payment
Search products
Upload profile image
```

Create a done checklist:

```
Requirement clarified
API designed
Input validation added
Success and failure responses handled
Unit tests written
Integration test added
Logs added
README updated
Deployment checked
Monitoring considered
```

Practice goal: think beyond "I wrote the code."

### Exercise 9: Incident Communication Drill

Imagine this production issue:

```
Users cannot submit orders because the payment API is timing out.
```

Write three messages:

1. Message to your engineering team
2. Message to your manager/product owner
3. Customer-facing status update

Practice goal: communicate clearly during pressure.

### Exercise 10: Cross-Team Handoff

Write a handoff note from backend to frontend for a new API.

Include:

```
Endpoint URL
Request body
Response body
Error response examples
Authentication requirement
Known limitations
Testing notes
```

Practice goal: reduce back-and-forth and prevent integration confusion.

## Module 47 Lab: Professional Communication And Collaboration

### Scenario

Your team is building an Order Management feature for an e-commerce system.

The feature includes:

```
React frontend
Spring Boot REST API
Oracle database
Kafka order-created event
External payment service
```

Your job is not to code the feature. Your job is to practice the professional skills an engineer uses around the feature.

## Lab Goals

By the end, you should have created:

```
1 README section
1 architecture diagram
1 technical decision record
1 stakeholder explanation
3 code review comments
1 feature ownership checklist
1 handoff note
```

## Task 1: Write A README Section

Create a README section for the Order Management feature.

Include:

```
Feature purpose
Main API endpoint
Required environment variables
How to run tests
Common troubleshooting notes
```

Example structure:

```
## Order Management

This feature allows users to create and track orders.

### API

POST /api/orders

### Environment Variables

PAYMENT_API_URL=
KAFKA_BOOTSTRAP_SERVERS=
DATABASE_URL=

### Tests
```

```
mvn test
```

### Troubleshooting

*If order creation fails, check payment service availability and Kafka connectivity.*

## Task 2: Draw An Architecture Diagram

Create a simple text diagram showing the flow:

```
User
|
v
React Frontend
|
v
Spring Boot Order API
|
+--> Oracle Database
|
+--> External Payment Service
|
+--> Kafka Topic: order-created
      |
      v
      Notification Service
```

Then answer:

Which part is synchronous?  
Which part is asynchronous?  
What could fail?  
What should be monitored?

## Task 3: Write A Technical Decision Record

Write a short decision record for this decision:

Should the Order API call the Notification Service directly, or publish an order-created event to Kafka?

Use this format:

# Decision: Use Kafka for Order Notifications

## Context

When an order is created, customers need to receive a notification.

## Decision

Use Kafka to publish an order-created event.

## ## Reasons

...

## ## Tradeoffs

...

## ## Alternatives Considered

...

### Task 4: Explain The Tradeoff To A Stakeholder

Write a non-technical explanation of why the team chose Kafka.

Keep it under 5 sentences.

Example tone:

Instead of making the order system wait for the notification system, we let the order system publish an event after the order is created. This keeps order creation faster and more reliable if notifications are temporarily delayed. The tradeoff is that we need extra monitoring to make sure notification messages are processed correctly.

### Task 5: Practice Code Review Feedback

Imagine you reviewed this code:

```
@PostMapping("/orders")
public ResponseEntity<String> createOrder(@RequestBody OrderRequest request) {
    if (request.getCustomerId() == null) {
        return ResponseEntity.badRequest().body("Missing customer");
    }

    if (request.getItems() == null || request.getItems().isEmpty()) {
        return ResponseEntity.badRequest().body("Missing items");
    }

    double total = 0;
    for (OrderItem item : request.getItems()) {
        total += item.getPrice() * item.getQuantity();
    }

    PaymentResponse payment = paymentClient.charge(request.getCustomerId(), total);

    Order order = new Order();
    order.setCustomerId(request.getCustomerId());
    order.setTotal(total);
    order.setStatus("CREATED");

    orderRepository.save(order);

    kafkaTemplate.send("order-created", order.getId().toString());
}
```

```
    return ResponseEntity.ok("Order created");  
}
```

Write 3 constructive review comments.

Focus on:

Controller responsibilities  
Validation  
Error handling  
Testing  
Transaction boundaries  
Kafka event payload

Example:

This controller currently handles validation, pricing, payment, persistence, and messaging. Could we move the business logic into an OrderService so the controller only handles HTTP request and response concerns?

## Task 6: Create A Feature Ownership Checklist

Create a checklist for when this feature is truly done.

Include:

Requirements clarified  
API contract documented  
Validation implemented  
Unit tests written  
Integration tests written  
Kafka message verified  
Payment failure handled  
Logs added  
Monitoring considered  
README updated  
Deployment verified

## Task 7: Write A Backend-To-Frontend Handoff Note

Write a handoff note for the frontend team.

Include:

Endpoint  
Request body  
Successful response  
Error responses

Loading-state advice  
Known limitations

Example:

### ## Backend Handoff: Create Order API

Endpoint:  
POST /api/orders

Request:

```
{
  "customerId": 1001,
  "items": [
    {
      "productId": 501,
      "quantity": 2
    }
  ]
}
```

Success:  
201 Created

Error Cases:  
400 - Missing customerId  
400 - Empty item list  
502 - Payment service unavailable

## Submission Checklist

Submit one document or markdown file containing:

README section  
Architecture diagram  
Decision record  
Stakeholder explanation  
Code review comments  
Feature ownership checklist  
Frontend handoff note

## Evaluation Criteria

You did well if your work is:

Clear  
Specific  
Professional  
Useful to another developer

Understandable to a non-technical stakeholder  
Focused on ownership, not just implementation

## Key Takeaway

Module 47 is about professional engineering behavior. Code matters, but your ability to communicate, collaborate, explain decisions, accept feedback, and own outcomes is what makes you effective on real teams.